

35.18. Changer de vitesse en vol : A accélère, B ralentit

Le bouton A ajoute 1 à la vitesse, B en enlève 1, entre 1 et 4 ; un appui compte une fois, même tenu longtemps. VITESSE et son chiffre s'affichent en haut.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : la vitesse change pendant le vol. Un appui sur **A** l'augmente de 1, un appui sur **B** la baisse de 1. Elle reste entre 1 et 4 : `vitesse < 4` avant d'ajouter, `vitesse > 1` avant d'enlever.

Le piège : un appui dure plusieurs images. Même un appui rapide garde le bouton enfoncé pendant 5 ou 6 images. Si on écrivait seulement `if (bouton(A)) vitesse++;`, la vitesse monterait de 1 à **chaque image** : de 1 à 4 en un clin d'œil.

La solution : se souvenir de l'image d'avant. La variable `aAvant` vaut 1 si A était déjà enfoncé à l'image précédente, 0 sinon. On n'ajoute que si A est enfoncé **maintenant** et ne l'était **pas avant** : c'est le **début** de l'appui, et il n'arrive qu'une fois. Puis on note où en est A pour l'image suivante. `bAvant` fait pareil pour B.

Déroulé d'un appui sur A de 5 images, vitesse 2 : image 1 — A enfoncé, `aAvant` = 0 → vitesse 3, `aAvant` = 1 ; images 2 à 5 — A enfoncé, mais `aAvant` = 1 → rien ; image 6 — A lâché → `aAvant` = 0. Un appui, +1.

L'affichage : `texte(1, 0, "VITESSE")` une fois, avant la boucle ; puis à chaque image `nombre(9, 0, vitesse, 1)` écrit son chiffre (1 chiffre) juste après.

À toi : monte la limite à 6, ou fais démarrer l'avion à la vitesse 1.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <sprite16>` → [35.2. La fonction sprite16\(\) — un personnage de 16 × 16](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **967 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	75 o	← la console, à l'allumage — la mise en route de la console
main()	311 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case

```

EcrireTexte      10 o ← texte() – écrit un texte, lettre par lettre
EcrireNombre    33 o ← nombre() – écrit un nombre en chiffres
RangerLutins    11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles     76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte     14 o ← démarrage – vide les deux cartes de l'écran
LireManette     37 o ← bouton() – lit les huit boutons
Tuiles          304 o ← CopierTuiles – 15 lettres sur 43, et 4 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert – la petite routine qui copie les lutins
"VITESSE"       7 o ← texte() – un texte gravé

```

Lettres gravées : EISTV0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite()   × 4   104 o sur place, dans main()
bouton()   × 6   30 o sur place, dans main() + LireManette (37 o)
texte()    × 1   11 o sur place, dans main() + EcrireTexte (10 o)
nombre()   × 1   10 o sur place, dans main() + EcrireNombre (33 o)
image()    × 1    6 o sur place, dans main() + AttendreImage (28 o)
sprite16() × 1    0 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// Changer de vitesse en vol : A accélère, B ralentit, entre 1 et 4.

#include "personnages.cpp" // la liste : AVION
#include <sprite16>        // sprite16() : quatre lutins en carré
#include <bouton>           // bouton() : lit un bouton de la manette
#include <texte>            // texte() : le mot VITESSE
#include <nombre>           // nombre() : le chiffre de la vitesse

```

2. UNE VARIABLE — VITESSE

```
uint8_t vitesse = 2; // pixels par image, de 1 à 4
```

3. UNE VARIABLE — AAVANT

```
uint8_t aAvant = 0; // 1 si A était déjà enfoncé à l'image d'avant
```

4. UNE VARIABLE — BAVANT

```
uint8_t bAvant = 0; // pareil pour B
```

5. UNE VARIABLE — PX

```
uint8_t px = 72;
```

6. UNE VARIABLE — PY

```
uint8_t py = 64;
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(1, 0, "VITESSE");

    while (true) {
        image();

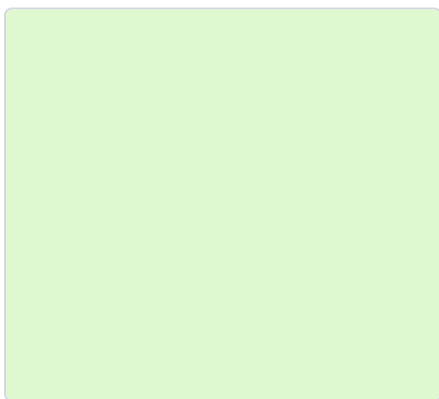
        // A : seulement au DÉBUT de l'appui (enfoncé maintenant, pas avant).
        if (bouton(A)) {
            if (aAvant == 0 && vitesse < 4) vitesse++;
            aAvant = 1;
        } else {
            aAvant = 0;
        }
        // B : pareil, pour ralentir.
        if (bouton(B)) {
            if (bAvant == 0 && vitesse > 1) vitesse--;
            bAvant = 1;
        } else {
            bAvant = 0;
        }

        if (bouton(DROITE) && px + vitesse <= 144) px = px + vitesse;
        if (bouton(GAUCHE) && px >= vitesse) px = px - vitesse;
        if (bouton(BAS) && py + vitesse <= 128) py = py + vitesse;
        if (bouton(HAUT) && py >= vitesse) py = py - vitesse;

        sprite16(0, px, py, AVION);
        nombre(9, 0, vitesse, 1); // le chiffre, juste après VITESSE
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé



B — l'écran a changé



DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

VITESSE 2 en haut ; chaque appui sur A fait monter le chiffre (jusqu'à 4) et l'avion va plus vite ; chaque appui sur B le fait descendre (jusqu'à 1).

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ VITESSE 2 s'affiche en haut (« VITESSE 2 »)
- ✓ un appui sur A, tenu 6 images : +1, pas +6 (vitesse 3)
- ✓ A ne dépasse pas 4
- ✓ B ne descend pas sous 1
- ✓ le chiffre affiché suit

Ensuite : **35.19. Plus lent qu'un pixel par image : la lenteur** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++